

COMANDI:

SINTASSI:

$C : \text{Skip} \mid x := e \mid \text{While } B \text{ do } C \mid \text{If } B \text{ do } C, \text{Else } C_2$

$d; C$

Blocco (deve Essere delimitato) $\Rightarrow$ procedure Esempio

BLOCCHI IN-LINE $\Rightarrow$ Senza Nome (permette di Creare un Ambiente locale)

(δ)

MEMORIE $\Rightarrow$ Associazioni Tra Locazioni e Valori Memorizzati.

e Sono Legami dinamici (CAMBIA VALORE NELLA LOCAZIONE DURANTE ESECUZIONE)

Sono MODIFICHE IREVERSIBILI (una Direzione)
Quelle che Subisce La Memoria

(ρ)

AMBIENTE $\Rightarrow$ Insieme di Associazioni Tra Identificatori ed Oggetti Denotabili (costanti e locazioni)

Vengono Associati / **BINDING STATICI**, che non Cambiano durante l'ESECUZIONE dei Comandi

Subiscono delle **Trasformazioni REVERSIBILI** (esistono dei Meccanismi Automatici per la Manipolazione dell'Ambiente)

BLOCCO:

SCOPO $\Rightarrow$ Per CREARE Ambienti Locali

Esistono METODI di Allocazioni di BLOCCHI diversa, che ad esempio possono come NO permettere la RICORSIONE

BLOCCHI SENZA-NOME / INLINE:

PROCEDURE: Blocchi con NOME che possiamo Richiamare

VISIBILITÀ:

Visibilità di una dichiarazione va di pari passo con la dichiarazione di un Riferimento per un Identificatore

Occorrenze libere di un Identificatore Sono definiti dalla dichiarazione

Qual'è la dichiarazione che NE definisce il SIGNIFICATO della DICHIARAZIONE

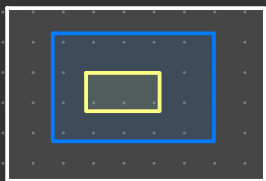
CONCETTO DI BLOCCO:

Devono Essere ANNOTATI INTERAMENTE



Non Può Essere devo Chiudere Prima Quello Precedente Per Aprire uno Nuovo

Passano Essere Solamente ANNIDATI Tra di loro



Regole STANDARD di **Visibilità di una DICHIARAZIONE**
locale è visibile (SCOPE) nel BLOCCO in cui è INSERITA ed
in TUTTI i Blocchi Annidati

Ma Nel caso in cui Andassimo a dichiarare un NUOVO
identificatore con Stesso Nome allora Verrebbe **SOVRASCRITTO**
TEMPORANEAMENTE (ed una volta chiuso il BLOCCO Tornerà
alla dichiarazione precedente)

①

ESEMPIO:

```
{const i: int = 3;
```

```
{ var i: real = 12;
```

```
  ...  
  i  
  ...  
}
```

ha SOVRASCRITTO LA **PRECEDENTE DICHIARAZIONE**

Scope della Variabile **i REALE**
(validità della **i REALE**)

```
i ...  
}
```

②

Scope della **i CONST** ha un BUCO, ① e ② visto che è
stata SOVRASCRITTA

Si VEDE Sempre e Solo Verso l'INTERNO MAI
Verso l'ESTERNO

SCOPE:

Area di Testo del CODICE Nel Quale tutte le occorrenze^E libere di un Identif. Sono LEGATE alla Stessa dichiarazione

Concetto di SPAZIO e NON di TEMPO

SCOPING STATICO $\Rightarrow$ Ambiente di riferimento degli identificatori
è STABILITO a Tempo di Compilazione

Blocco che lo CONTIENE SINTATTICAMENTE

Non tutti i linguaggi lo CONCEDONO

Classificazione degli ID:

LOCAL $\Rightarrow$ (Al Blocco) Quando è SCRITTA Nel Blocco

NON LOCAL $\Rightarrow$ Quando è VISIBILE al Blocco ma SCRITTA
in un BLOCCO DIFFERENTE (a lvl più ESTERNO)

Non ACCORGERSI DELLA RIDEFINIZIONE è un ERRORE Comune

/// GLOBALI $\Rightarrow$ Quando è VISIBILE a TUTTI I BLOCCHI

AMBIENTE LOCALE:

Insieme delle Associazioni locali del BLOCCO:

/// dichiarazioni locali

/// Parametri formali (PROCEDURE)

$add(m,n) \Rightarrow m,n$ fanno parte dell' AMBIENTE

AMBIENTE NON LOCALE:

Visibile ma non definito localmente

AMBIENTE GLOBALE:

Ambiente visibile all'intero Programma

TEMPO DI VITA:

Corrisponde al Tempo in cui la locazione è ASSOCIATA all'ID.

Consiste nel TEMPO di Esecuzione Nel Quale tutte le occor. libere di un ID si riferiscono alla Stessa Locazione

SCOPE:

Area di Codice

Dichiarazione

```
{ const i: int = 3;  
  { var i: Real = 1.2;  
    ... i ...  
  }  
  ... i ...  
}
```

TEMPO DI VITA:

Tempo di Esecuzione

Locazione

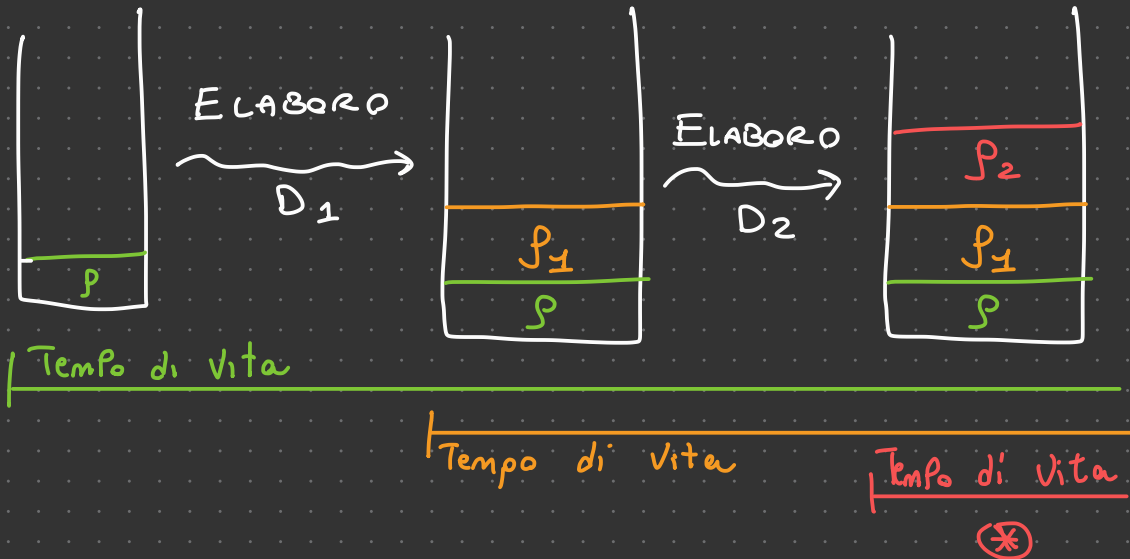
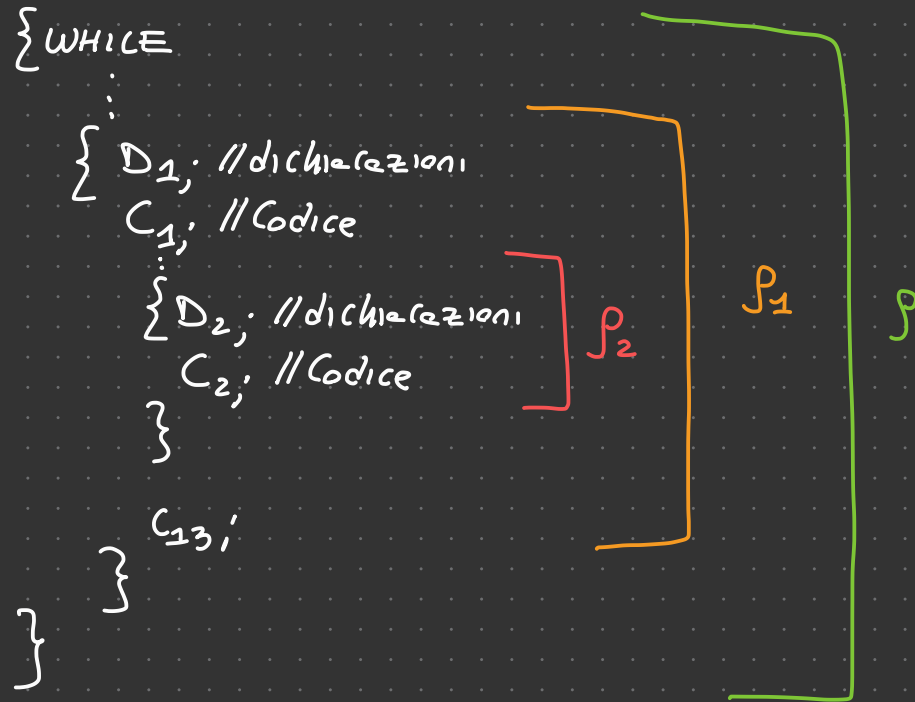
Tempo di vita di CONST di NON ha Intersezioni altrimenti NON sarei in Grada di TORNARE indietro e Viene Salvato da un ALTRA PARTE

→ Tempo di vita = Scope in questo caso

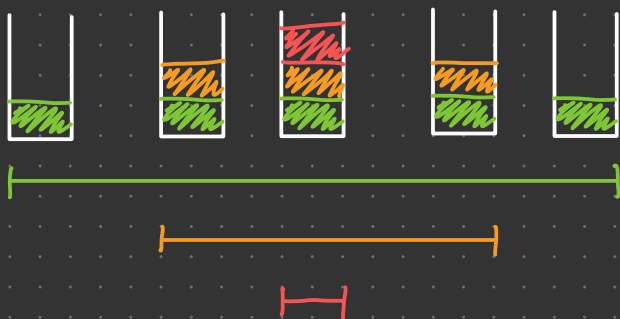
→ SCOPE di CONST i

SCOPE dipende Solo dalle REGOLE Mentre TEMPO di vita,
dipende da Sulle Implementative RIGUARDO alla MEMORIA

ESEMPIO:



(*) Che poi Viene subito deallocato e quindi si conclude subito



Quando Sono uscito dal WHILE

SEMANTICA $D; C$:

$$FI(d; c) = FI(d) \cup (FI(c) \setminus DI(d))$$

Var $x: int = 3 * y$; D } y è LIBERA $\Rightarrow FI$ ma x è
 $x := z + x$; C } DEFINITA

$FI(c) = \{z, x\}$ ma Nel

Blocco $FI(d; c) = \{z\}$

$$DI(d; c) = DI(d) \cup DI(c)$$

SEMANTICA STATICA dice Quando un Blocco è BEN formato

ma Non ho Vincoli visto che devo Verificare che sia i
 COMANDI e dichiarazioni Siano BEN FORMATI

Ambiente Costituito da d

$$\frac{\Delta \vdash d \rightarrow \Delta_0}{\Delta \vdash d; c \rightarrow}$$

$\Delta[\Delta_0] \vdash c$

Rispettare i Vincoli Anche dell' Ambiente Aggiornato

Ricordarsi di Aggiornarlo

SEMANTICA DINAMICA:

$\rho \vdash \langle c, \delta \rangle \rightarrow \langle c, \delta' \rangle$ Potenzialmente Modificata

① ELABORO d :

$\rho \vdash \langle d, \delta \rangle \xrightarrow{*} \langle \rho_0, \delta' \rangle$ AMBIENTE DINAMICO COSTRUITO DA d

$\rho \vdash \langle d; c, \delta \rangle \rightarrow \langle \rho_0; c, \delta' \rangle$

② Esegui c

$\rho[\rho_0] \vdash \langle c, \delta \rangle \rightarrow \langle c', \delta' \rangle$

$\rho \vdash \rho_0; c, \delta' \rightarrow \langle \rho_0, c', \delta' \rangle$

③ COMANDO HA TERMINATO $\rightarrow$ ANCHE BLOCCO HA TERMINATO

$\rho[\rho_0] \vdash \langle c, \delta \rangle \rightarrow \delta'$

$\rho \vdash \rho_0; c, \delta \rightarrow \delta'$

(Nota che ρ_0 è locale al Blocco)

ESECUZIONE ed EQUIVALENZA:

Exec: $Com \times Mem \rightarrow Mem$

$Exec(c, \delta) = \delta'$ sse $\emptyset \vdash \langle c, \delta \rangle \rightarrow^* \delta'$
Senza Ambiente EXT. 

EQUIVALENZA: $\equiv \subseteq Com \times Com$

$C_1, C_2 \in Com$

$C_1 \equiv C_2$ sse $\forall \delta \quad Exec(C_1, \delta) = Exec(C_2, \delta)$

Indipendentemente dalla Memoria ottengo lo Stesso Risultato, la funzione Calcolata Ψ di Fondamenti e per questo so che questa è la prima Equivalenza non decidibile per via del ciclo WHILE

Un EXEC può non Terminare (per ciclo WHILE) visto che il ciclo WHILE porta ad un ALTRO WHILE e se la guardia fosse Sempre Vera allora potenzialmente divergo/ non termino MAI

ESEMPIO SEMANTICA BLOCCO!

var x:int = 3; x := x + 1

d c

da dim

SEMANTICA STATICA:

$\vdash \text{Var } x:\text{int} = 3 : \Delta_0$

$\Delta_0 \vdash x := x + 1$

$\vdash \text{Var } x:\text{int} = 3; x := x + 1$

$\vdash 3:\text{int}$

$\vdash \text{Var } x:\text{int} = 3 : [\bar{x} \mapsto \text{int}_{\text{loc}}] \equiv \Delta_0$

$\Delta_0 \vdash x + 1 = \text{int}$

$\Delta_0 \equiv [\text{int}_{\text{loc}}] \vdash x := x + 1$

$\Delta_0(x) = \text{int}_{\text{loc}}$

$\Delta_0 \vdash x:\text{int}$

$\Delta_0 \vdash 1:\text{int}$

$\Delta_0(x) = \text{int}_{\text{loc}}$

$\Delta_0 \vdash x + 1:\text{int}$

Viene subito usato
l'AMBIENTE creato

SEMANTICA DINAMICA:

$$\vdash \langle d; C, \phi \rangle \longrightarrow ?$$

$\searrow \delta = \emptyset$

$$\textcircled{1} \quad \vdash \langle \text{Ver } x:\text{int} = 3, \delta \rangle \xrightarrow{*} ? \langle \rho_0, \delta' \rangle ?$$
$$\vdash \langle \text{Ver } x:\text{int} = 3; C \rangle \longrightarrow \langle \rho_0; C, \delta' \rangle$$

Elabora dichiarazione ma c'è già assioma (un valore)

$$\vdash \langle 3, \delta \rangle \longrightarrow$$
$$\vdash \langle \text{Ver } x:\text{int} = 3; C \rangle \longrightarrow \langle \underbrace{[x \mapsto Cx]}_{\rho_0}, \underbrace{[Cx \mapsto 3]}_{\delta'} \rangle$$

Tornando ad $\textcircled{1}$

$$\vdash \langle \text{Ver } x:\text{int} = 3; C \rangle \longrightarrow \langle \rho_0; \quad$$

$$\textcircled{2} \quad \rho_0 \vdash \langle x := x + 1, \delta' \rangle \longrightarrow ?$$